

# Timers and Hazard Rates

Controlling when events fire in AlgebraicABMs.jl

Simon Frost

2026-03-30

## 1 Overview

In AlgebraicABMs.jl, **timers** determine *when* rewrite rules fire. Each **ABMRule** pairs a rewriting rule with a timer that produces a hazard rate or waiting-time distribution. The framework supports eight timer types, ranging from simple constant rates to history-dependent closures.

This vignette demonstrates every timer type using small epidemiological models built on the **Graph** schema.

## 2 Setup

```
using AlgebraicABMs
using Catlab, AlgebraicRewriting
using Distributions: Exponential
using Random
Random.seed!(42)

TaskLocalRNG()
```

## 3 Helper: a reusable edge-removal rule

We define a minimal “infection” rule that removes an edge from a graph (representing consumption of a contact), and a “recovery” rule that removes a vertex. These will be reused with different timers throughout.

```
# Infection: remove an edge (S-I contact consumed)
L_inf = @acset Graph begin V=2; E=1; src=[1]; tgt=[2] end
I_inf = Graph(2)
R_inf = Graph(2)
inf_rule = Rule(
    homomorphism(I_inf, L_inf; initial=(V=[1,2],)),
    homomorphism(I_inf, R_inf; initial=(V=[1,2],)))

# Recovery: remove a vertex
L_rec = Graph(1)
I_rec = Graph(0)
R_rec = Graph(0)
rec_rule = Rule(homomorphism(I_rec, L_rec), id(I_rec))

# Small initial state: 6 vertices, 10 directed edges
init = @acset Graph begin
    V = 6
    E = 10
```

```

src = [1,1,2,2,3,3,4,4,5,6]
tgt = [2,3,3,4,5,6,5,6,6,1]
end
println("Initial state: $(nparts(init, :V)) vertices, $(nparts(init, :E)) edges")
Initial state: 6 vertices, 10 edges

```

## 4 1. DiscreteHazard — Fixed Waiting Time

A `DiscreteHazard` fires at an exact time offset from when it is enabled. Passing a number wraps it in a Dirac distribution.

**Epidemiological interpretation:** A fixed incubation period — every exposed individual becomes infectious after exactly 5 time units.

```

# Fixed incubation: edge removed at exactly t=5 from enablement
infection_discrete = ABMRule(:InfDiscrete, inf_rule, DiscreteHazard(5.0))

abm_disc = ABM([infection_discrete])
traj_disc = run!(abm_disc, deepcopy(init); maxtime=30.0)

println("DiscreteHazard events: $(length(traj_disc))")
for (i, (t, _, name, _)) in enumerate(traj_disc.events)
    i > 8 && break
    println("  t=$(round(t, digits=2)): $name")
end

[ Info: Stochastic scheduling algorithm ran out of events
DiscreteHazard events: 10
t=5.0: InfDiscrete
t=5.0: InfDiscrete
t=5.0: InfDiscrete
t=5.0: InfDiscrete
t=5.0: InfDiscrete
t=5.0: InfDiscrete
t=5.0: InfDiscrete
t=5.0: InfDiscrete

```

All events fire at multiples of 5.0 because each match is enabled at time 0 and fires exactly 5 units later. Subsequent matches enabled after a rewrite also wait exactly 5 units.

## 5 2. ContinuousHazard — Constant Exponential Rate

`ContinuousHazard` wraps a continuous distribution (typically `Exponential`). Passing a number `p` constructs `Exponential(p)` (mean waiting time `p`).

**Epidemiological interpretation:** Constant-rate infection — each contact transmits independently at rate  $1/0.3$  per unit time (mean 0.3).

```

infection_cont = ABMRule(:InfCont, inf_rule, ContinuousHazard(0.3))
recovery_cont = ABMRule(:RecCont, rec_rule, ContinuousHazard(Exponential(5.0)))

abm_cont = ABM([infection_cont, recovery_cont])
traj_cont = run!(abm_cont, deepcopy(init); maxtime=15.0)

println("ContinuousHazard events: $(length(traj_cont))")

```

```

for (i, (t, _, name, _)) in enumerate(traj_cont.events)
  i > 10 && break
  println(" t=$(round(t, digits=3)): $name")
end

[ Info: Stochastic scheduling algorithm ran out of events
ContinuousHazard events: 16
t=0.014: InfCont
t=0.034: InfCont
t=0.056: InfCont
t=0.147: InfCont
t=0.161: InfCont
t=0.166: InfCont
t=0.189: InfCont
t=0.352: InfCont
t=0.412: InfCont
t=0.742: InfCont

```

Events are irregularly spaced — each follows an exponential waiting time. This is the classic Gillespie-style stochastic simulation.

## 6 3. ClosureTime — Time-Varying Hazard

ClosureTime takes a function `clocktime → hazard_rate` (returning a distribution). The rate can depend on the absolute simulation clock but **not** on the state.

**Epidemiological interpretation:** Seasonal forcing — transmission peaks in winter (modeled as a sinusoidal rate).

```

# Seasonal rate: peaks at t=0,10,20,... (period 10)
seasonal_timer = ClosureTime(t -> Exponential(0.5 + 0.4 * cos(2 * t / 10.0)))

infection_seasonal = ABMRule(:InfSeasonal, inf_rule, seasonal_timer)
abm_season = ABM([infection_seasonal])
traj_season = run!(abm_season, deepcopy(init); maxtime=20.0)

println("ClosureTime (seasonal) events: $(length(traj_season))")
for (i, (t, _, name, _)) in enumerate(traj_season.events)
  i > 8 && break
  println(" t=$(round(t, digits=3)): $name")
end

[ Info: Stochastic scheduling algorithm ran out of events
ClosureTime (seasonal) events: 10
t=0.114: InfSeasonal
t=0.159: InfSeasonal
t=0.318: InfSeasonal
t=0.37: InfSeasonal
t=0.427: InfSeasonal
t=0.52: InfSeasonal
t=0.975: InfSeasonal
t=1.531: InfSeasonal

```

The rate oscillates, so events cluster around times when the hazard rate is highest.

## 7 4. ClosureState — State-Dependent Hazard

`ClosureState` takes a function `match_morphism → distribution`. The rate depends on the current match (and through it, the full state) but **not** on clock time.

**Epidemiological interpretation:** Density-dependent transmission — the infection rate for a given contact is proportional to the total number of edges (more contacts → faster spread).

```
# Rate proportional to total edge count
density_timer = ClosureState(m -> begin
  state = codom(m)
  n_edges = nparts(state, :E)
  Exponential(max(0.1, 10.0 / n_edges))
end)

infection_density = ABMRule(:InfDensity, inf_rule, density_timer)
abm_density = ABM([infection_density])
traj_density = run!(abm_density, deepcopy(init); maxtime=20.0)

println("ClosureState (density) events: $(length(traj_density))")
for (i, (t, _, name, _)) in enumerate(traj_density.events)
  i > 8 && break
  println("  t=$(round(t, digits=3)): $name")
end
```

```
[ Info: Stochastic scheduling algorithm ran out of events
ClosureState (density) events: 10
t=0.042: InfDensity
t=0.259: InfDensity
t=0.265: InfDensity
t=0.365: InfDensity
t=0.454: InfDensity
t=0.652: InfDensity
t=0.717: InfDensity
t=0.952: InfDensity
```

As edges are removed, the remaining edges have a higher mean waiting time (slower rate), creating a decelerating epidemic.

## 8 5. ClosureParams — Parameter-Dependent Hazard

`ClosureParams` takes a function `(match_morphism, params) → distribution`. The `params` come from `ABM(...; params=...)` and allow external parameter sweeps without redefining rules.

**Epidemiological interpretation:** Transmission rate set by an external parameter (e.g., from calibration or scenario analysis).

```
param_timer = ClosureParams((m, params) -> Exponential(params.beta))

infection_param = ABMRule(:InfParam, inf_rule, param_timer)
abm_param = ABM([infection_param]; params=(beta=0.5,))
traj_param = run!(abm_param, deepcopy(init); maxtime=15.0)

println("ClosureParams (=0.5) events: $(length(traj_param))")
for (i, (t, _, name, _)) in enumerate(traj_param.events)
  i > 6 && break
end
```

```

    println("  t=$(round(t, digits=3)): $name")
end

[ Info: Stochastic scheduling algorithm ran out of events
ClosureParams (=0.5) events: 10
  t=0.003: InfParam
  t=0.035: InfParam
  t=0.092: InfParam
  t=0.146: InfParam
  t=0.26: InfParam
  t=0.335: InfParam

```

The same rule can be reused with different params via `run_scenarios`.

## 9 6. FullClosure — Time + State Dependent

`FullClosure` takes  $(\text{match\_morphism}, \text{clocktime}) \rightarrow \text{distribution}$ . This is the most flexible non-parameterized timer: the hazard depends on both the current state and the clock.

**Epidemiological interpretation:** A declining epidemic — transmission depends on how many edges remain AND decays over time (e.g., behavioral adaptation).

```

full_timer = FullClosure((m, t) -> begin
  state = codom(m)
  n_edges = nparts(state, :E)
  decay = exp(-0.1 * t)
  Exponential(max(0.1, 5.0 / (n_edges * decay)))
end)

infection_full = ABMRule(:InfFull, inf_rule, full_timer)
abm_full = ABM([infection_full])
traj_full = run!(abm_full, deepcopy(init); maxtime=25.0)

println("FullClosure events: $(length(traj_full))")
for (i, (t, _, name, _)) in enumerate(traj_full.events)
  i > 8 && break
  println("  t=$(round(t, digits=3)): $name")
end

[ Info: Stochastic scheduling algorithm ran out of events
FullClosure events: 10
  t=0.009: InfFull
  t=0.025: InfFull
  t=0.073: InfFull
  t=0.243: InfFull
  t=0.277: InfFull
  t=0.364: InfFull
  t=0.503: InfFull
  t=0.827: InfFull

```

Early events are fast (many edges, little decay); late events slow down as both edges deplete and the time-decay factor kicks in.

## 10 7. FullClosureParams — Time + State + Params

FullClosureParams takes (match\_morphism, clocktime, params) → distribution. The most general parameterized timer.

**Epidemiological interpretation:** Seasonal transmission with an externally-set baseline rate.

```
fullparam_timer = FullClosureParams((m, t, p) -> begin
    seasonal = 1.0 + 0.5 * sin(2 * t / p.period)
    Exponential(p.base_rate / seasonal)
end)

infection_fp = ABMRule(:InfFullParam, inf_rule, fullparam_timer)
abm_fp = ABM([infection_fp]; params=(base_rate=0.5, period=8.0))
traj_fp = run!(abm_fp, deepcopy(init); maxtime=20.0)

println("FullClosureParams events: $(length(traj_fp))")
for (i, (t, _, name, _)) in enumerate(traj_fp.events)
    i > 8 && break
    println("  t=$(round(t, digits=3)): $name")
end
```

```
[ Info: Stochastic scheduling algorithm ran out of events
FullClosureParams events: 10
  t=0.021: InfFullParam
  t=0.07: InfFullParam
  t=0.12: InfFullParam
  t=0.134: InfFullParam
  t=0.313: InfFullParam
  t=0.374: InfFullParam
  t=0.511: InfFullParam
  t=0.56: InfFullParam
```

Both base\_rate and period can be varied across scenarios.

## 11 8. ClosureHistory — History-Sensitive Hazard

ClosureHistory takes (match\_morphism, clocktime, traj) → distribution. The trajectory contains all past events, enabling rates that depend on the full history.

**Epidemiological interpretation:** Waning immunity — as more events accumulate, the rate slows down (modeling saturation or exhaustion of susceptible contacts).

```
using Catlab.CategoricalAlgebra.CSets: BitSetParts
@present SchGrphH(FreeSchema) begin
    V::Ob; E::Ob; src::Hom(E,V); tgt::Hom(E,V)
end
@acset_type GrphH(SchGrphH, part_type=BitSetParts)

L_h = @acset GrphH begin V=2; E=1; src=[1]; tgt=[2] end
I_h = @acset GrphH begin V=2 end
R_h = @acset GrphH begin V=2 end
inf_rule_h = Rule(
    homomorphism(I_h, L_h; initial=(V=[1,2])),
    homomorphism(I_h, R_h; initial=(V=[1,2])))

history_timer = ClosureHistory((m, t, traj) -> begin
```

```

    n_past = isnothing(traj) ? 0 : length(traj)
    Exponential(0.5 * (1 + n_past))
end)

infection_hist = ABMRule(:InfHistory, inf_rule_h, history_timer)
abm_hist = ABM([infection_hist])

init_h = @acset GrphH begin
    V = 5
    E = 8
    src = [1,1,2,2,3,3,4,5]
    tgt = [2,3,3,4,5,4,5,1]
end

traj_hist = run!(abm_hist, deepcopy(init_h); maxtime=30.0)

println("ClosureHistory events: $(length(traj_hist))")
times = [e[1] for e in traj_hist.events]
for (i, (t, _, name, _)) in enumerate(traj_hist.events)
    i > 8 && break
    println("  t=$(round(t, digits=3)): $name")
end

if length(times) >= 2
    intervals = diff(times)
    println("\nInter-event intervals (first 5): ",
        join(round.(intervals[1:min(5,end)], digits=3), ", "))
    println("Intervals grow as history accumulates → waning immunity effect")
end

[ Info: Stochastic scheduling algorithm ran out of events
ClosureHistory events: 8
  t=0.187: InfHistory
  t=0.193: InfHistory
  t=0.211: InfHistory
  t=0.344: InfHistory
  t=0.584: InfHistory
  t=0.598: InfHistory
  t=0.791: InfHistory
  t=2.732: InfHistory

```

Inter-event intervals (first 5): 0.006, 0.018, 0.134, 0.239, 0.014

Intervals grow as history accumulates → waning immunity effect

Each successive event increases the mean waiting time, simulating population-level saturation.

## 12 Summary

| Timer Type          | Depends On       | Use Case                           |
|---------------------|------------------|------------------------------------|
| DiscreteHazard(t)   | Nothing (fixed)  | Fixed incubation, scheduled events |
| ContinuousHazard(d) | Nothing (random) | Constant-rate Gillespie dynamics   |

| Timer Type                        | Depends On                | Use Case                             |
|-----------------------------------|---------------------------|--------------------------------------|
| <code>ClosureTime(f)</code>       | Clock time                | Seasonal forcing, time-varying rates |
| <code>ClosureState(f)</code>      | Match/state               | Density-dependent transmission       |
| <code>ClosureParams(f)</code>     | Match + params            | Externally parameterized rates       |
| <code>FullClosure(f)</code>       | Match + time              | State and time dependent             |
| <code>FullClosureParams(f)</code> | Match + time + params     | General parameterized model          |
| <code>ClosureHistory(f)</code>    | Match + time + trajectory | History-sensitive (waning immunity)  |

The choice of timer controls the **stochastic dynamics** without changing the rewriting rules. This separation of *what happens* (rules) from *when it happens* (timers) is a key design principle of AlgebraicABMs.jl.